# Progress Report

Discrete State-Space Models and Expectation-Maximization for Cell-State Trajectories

Arnab Kumar Ghosh · Supervisor: Prashant Singh · MSc Computer Science, Uppsala University · Weeks 3–4 · September 29, 2026

## Overview

In weeks 3–4 I moved from the model definition to the algorithms. The open question from the last report (how to build the observation sequence) was discussed with the supervisor in the meeting on 22 September, but I still have questions about it (Section 7). Progress against the plan from the last report:

- Forward and backward algorithms with per-step scaling (Rabiner Problem 1): **done**, on paper and in code.
- Posteriors $\gamma_t(i)$ and $\xi_t(i,j)$, i.e. the E-step: **done** in code.
- Baum–Welch re-estimation (Rabiner Problem 3) for discrete emissions: **done** in code, with checks.
- Sequence definition: discussed with the supervisor; still open, see Section 7.
- Bilmes (1998) and the Gaussian-emission M-step: **not yet**.
- Data received from the biology group, with a read-me describing the dataset.
- Read the previous student's internship report and identified why its results went wrong (Section 6).

The toy implementation is a small NumPy script: $N = 3$ states, $M = 4$ discrete symbols, known parameters $\lambda = (A, B, \pi)$, a sampler to generate sequences, and one function per algorithm. The rest of this report summarises what I understand so far and how each part connects to the project.

## 1 The forward algorithm (Rabiner, Problem 1)

### What I learned

The hidden state at time $t$ is $s_t$, the observation sequence is $O = o_1 \cdots o_T$, and the model is $\lambda = (A, B, \pi)$ with

$$\begin{aligned} A[i,j] &= P(s_{t+1} = j \mid s_t = i), && \text{transition: from state } i \text{ to state } j, \\ B[j,k] &= P(o_t = k \mid s_t = j), && \text{emission: state } j \text{ produces symbol } k, \\ \pi_i &= P(s_1 = i). && \text{start in state } i. \end{aligned}$$

In the code these are the arrays `A`, `B` and `pi`.

The forward variable is the probability of seeing the first $t$ observations and ending in state $i$:

$$\alpha_t(i) = P(o_1 \cdots o_t,\ s_t = i \mid \lambda).$$

It is filled in one time step at a time, and the last row gives the likelihood of the whole sequence:

$$\begin{aligned} \alpha_1(i) &= \pi_i B[i, o_1], \\ \alpha_t(j) &= \Big[\sum_{i=1}^{N} \alpha_{t-1}(i) A[i,j]\Big] B[j, o_t], \\ P(O \mid \lambda) &= \sum_{i=1}^{N} \alpha_T(i). \end{aligned}$$

The sum inside the bracket is "all the ways to arrive in state $j$", and the factor $B[j, o_t]$ is "and then see the new observation". This replaces the sum over all $N^T$ state paths with a table of $N \times T$ numbers.

### In the project

$P(O \mid \lambda)$ is the likelihood of the data under the current parameters. It is the quantity that must never decrease during Baum–Welch, so the forward algorithm is also the tool for checking that training is working.

## 2 The backward algorithm and the posteriors (E-step)

### What I learned

The backward variable is the probability of the rest of the sequence, given that the state at time $t$ is $i$:

$$\beta_t(i) = P(o_{t+1} \cdots o_T \mid s_t = i, \lambda).$$

It is filled in from the end:

$$\begin{aligned} \beta_T(i) &= 1, \\ \beta_t(i) &= \sum_{j=1}^{N} A[i,j]\, B[j, o_{t+1}]\, \beta_{t+1}(j). \end{aligned}$$

Together, $\alpha$ and $\beta$ give the two posteriors needed by EM. $\gamma_t(i)$ is the probability of being in state $i$ at time $t$, and $\xi_t(i,j)$ the probability of being in state $i$ at time $t$ and in state $j$ at time $t+1$, both given the whole sequence:

$$\begin{aligned} \gamma_t(i) &= \frac{\alpha_t(i)\,\beta_t(i)}{P(O \mid \lambda)}, \\ \xi_t(i,j) &= \frac{\alpha_t(i)\, A[i,j]\, B[j, o_{t+1}]\, \beta_{t+1}(j)}{P(O \mid \lambda)}. \end{aligned}$$

$\alpha$ carries the evidence from the past, $\beta$ the evidence from the future, and $\gamma$ combines both. In the code, $\gamma$ rows sum to 1, every $\xi_t$ slice sums to 1, and $\sum_j \xi_t(i,j) = \gamma_t(i)$; these are asserted after every run.

**In the project**

$\gamma_t(i)$ is, for each cell, the probability of being a Macrophage, Transitional or Mural-like cell. On the toy data, $\arg\max_i \gamma_t(i)$ recovers the true state at most time steps, which is how cells could be labelled later.

## 3 Scaling (the numerical issue from the last report)

**What I learned**

Every $\alpha_t(i)$ is a product of $t$ probabilities, so it underflows to zero for long sequences. The standard fix is to divide each row of $\alpha$ by its sum and remember that sum as the scale factor:

$$c_t = \sum_{i=1}^{N} \alpha_t(i),$$
$$\hat{\alpha}_t(i) = \frac{\alpha_t(i)}{c_t},$$
$$\log P(O \mid \lambda) = \sum_{t=1}^{T} \log c_t.$$

$\beta$ is scaled with the same factors, but row $t$ is divided by $c_{t+1}$, because $\beta_t$ covers the future observations $o_{t+1} \cdots o_T$ while $\alpha_t$ already covers $o_1 \cdots o_t$. Then $\hat{\alpha}_t(i)\hat{\beta}_t(i)$ sums to 1 at every $t$ and $\gamma$ is simply $\hat{\alpha}\hat{\beta}$, so the posteriors are unchanged. In the code the forward and backward passes each compute the log-likelihood independently and the two values agree to machine precision, which is the check that the scaling is consistent.

**In the project**

The implementation uses scaling from the start, as agreed with the supervisor, so the log-likelihood check is always well defined.

## 4 Baum–Welch re-estimation (Rabiner, Problem 3)

**What I learned**

The M-step turns the posteriors into new parameters. Every update is a normalised expected count: expected number of transitions from $i$ to $j$ over expected number of visits to $i$, and expected number of visits to $j$ that emit symbol $k$ over expected number of visits to $j$:

$$\bar{\pi}_i = \gamma_1(i),$$
$$\bar{A}[i,j] = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)},$$
$$\bar{B}[j,k] = \frac{\sum_{t:\, o_t = k} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}.$$

Baum–Welch repeats E-step and M-step until the log-likelihood stops increasing. I tested it on a sequence of 2000 toy observations sampled from the true $\lambda$, starting from five different random parameter sets (Figure 1). Panel (a) shows the log-likelihood after every iteration. Every curve rises monotonically: it never goes down, which the code asserts. The dashed line is the log-likelihood under the true parameters ($-2698.1$). Four of the five starts end slightly *above* this line (about $-2688$). This is not an error. The true parameters describe the generating process, while Baum–Welch tunes the parameters to this one finite sample, including its random noise, so it can fit the sample a little better. The fifth start (start 2) ends lower, at $-2697.5$: it climbed a different hill and stopped at a local optimum.

Panel (b) shows why the fitted $B$ does not look like the true one even for a good start. The middle matrix is what Baum–Welch returns for start 4: the rows are the true rows in a different order. This is label switching. The state names S1, S2, S3 are arbitrary, and renaming the states gives exactly the same likelihood, so Baum–Welch cannot know which fitted state should be called S1. Once the rows are re-ordered to match (right matrix) the fitted $B$ is close to the true $B$. So two things are needed to recover the generating parameters: several random restarts, keeping the best log-likelihood, to avoid local optima such as start 2; and matching the state labels before comparing with the truth.

**In the project**

This is the training loop the project needs. The $\bar{A}[i,j]$ and $\bar{\pi}_i$ updates carry over unchanged; only $\bar{B}[j,k]$ has to be replaced by the updates for the Gaussian mean $\mu_j$ and covariance $U_j$ from Bilmes. The local-optimum behaviour is a warning for the real data: several restarts and a comparison with the known labels will be necessary.

## 5 The dataset

**What I learned**

The data arrived as `adata_final.h5ad`, a combination of two single-cell sets (ss2 and ss3). ss2 covers day 21 and healthy tissue; ss3 covers days 7, 14, 21 (few cells) and healthy. So days 7 and 14 come only from ss3, while day 21 comes mostly from ss2: a difference between day 7/14 and day 21 can be partly protocol rather than biology. The read-me notes that the

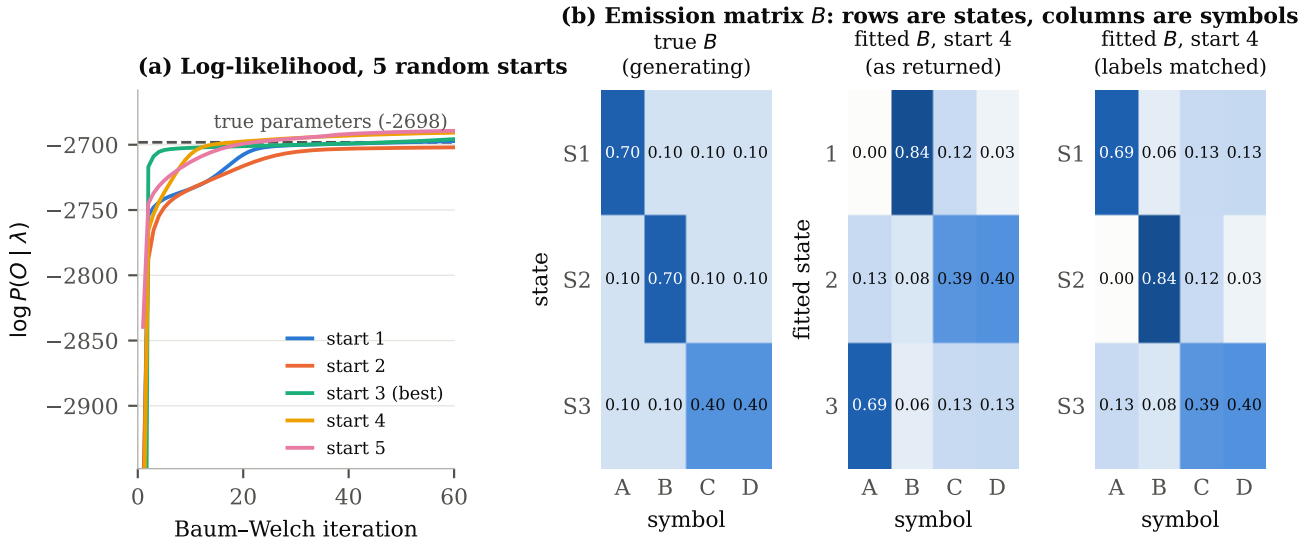



Figure 1: Baum–Welch on 2000 toy observations. (a) Log-likelihood per iteration for five random starts; the dashed line is the value under the true parameters. Every curve rises monotonically, four starts end slightly above the true value and one (start 2) stops at a lower local optimum. (b) The emission matrix $B$: the true $B$, the fitted $B$ for start 4 as returned (rows in a different order: label switching), and the same matrix after re-ordering the rows to match the true states. Produced by `plot_baum_welch.py`, 200 iterations per start.

pseudotime looks the same on ss2 alone and on ss2+ss3, which suggests the batch effect is not dominant, but I will keep the batch label per cell. The three states of the model map onto labels that already exist in the data:

| Model state | Label in the data | Note |
|---|---|---|
| Macrophage | `cell_type` = "Macrophages" | Starting population after injury |
| Transitional | `cluster` = 2 (PDGFRb$^+$ macrophages) | Almost absent at day 7 and 14, many at day 21; high unspliced-mRNA fraction, a sign of active reprogramming |
| Mural-like | `cell_type` = "Mural cells", `cluster` = 8 | Marker genes include *Pdgfrb*, *Acta2*, *Myh11*, *Notch3* |

### In the project

The existing labels will be used only as an external check of the fitted states, not as training input. The observation vector $O$ for each cell will be the expression of the 50 genes listed in the read-me: the top 25 marker genes of cluster 2 (PDGFRb$^+$ macrophages) and the top 25 of cluster 8 (mural cells). A Gaussian over all genes is not feasible, and these 50 were agreed with the supervisor as the starting point.

## 6 The previous student's report

### What I learned

I read the internship report of the previous student. He built the same three algorithms (forward–backward, Viterbi, Baum–Welch) in Python. To get observations, he ran $k$-means on three marker genes and gave each cell one of three labels. Then he put all cells in one long list, sorted by day, and used that list as the observation sequence. He describes two problems he could not solve:

1. With a random start, Baum–Welch ended in a poor local maximum and the result made no biological sense. So he set $\pi$, $A$ and $B$ by hand to match the paper. I saw the same thing on my toy data (Section 4). The problem with his fix is that if you start from the answer you expect, you mostly get that answer back. Also, any entry set to 0 in $A$ stays 0 forever.
2. His result said 100 % of cells are intermediate on day 7 and 100 % are mural on day 21. The paper says about 10 % and 20 %. The reason is how the sequence was built. Each cell in the list counts as one time step, so there are thousands of steps but the state only changes a few times (at the day boundaries). Baum–Welch therefore learns that changing state is almost impossible ($A$ has 0.993, 0.999 and 1.0 on the diagonal). Viterbi then gives the same state to every cell in a day block. So the proportions can only be 0 % or 100 %. This comes from the construction, not from the biology.

### In the project

This supports the direction agreed with the supervisor. In the population-level model, one step of $A$ is one week, not one cell. Cells within a day are a mixture, so proportions can be any value. Emissions are Gaussians on real expression values, not $k$-means labels. Per-cell states come from $\gamma$, not from one Viterbi path. I will also use several random restarts and start from the data, not from the expected answer.

## 7 What is still unclear to me

- Sequence definition. scRNA-seq measures each cell only once, so no single cell can be followed over time, and I do not yet see what the observation sequence $o_1 \cdots o_T$ should be. Ordering all cells by day makes every cell a time step, which is the construction that went wrong in the previous student's report (Section 6). The supervisor suggested a population-level view, where one step of $A$ is one week and the cells on a day are a mixture of the states, but I am not yet sure how this is written as an HMM likelihood and how it is fitted. This is my main question for the next meeting.
- The Gaussian-emission M-step (Bilmes): how the $\bar{B}[j,k]$ update above becomes updates for $\mu_j$ and $U_j$, and how to keep $U_j$ well conditioned when a state has few cells.
- How to deal with local optima in practice: how many restarts, and how to pick between fits that differ only by label switching.
- Which matrix in `adata_final.h5ad` to use for the 50 genes. The scaled `.X` matrix contains negative values, while `.raw.X` is log-normalised and non-negative; which one is suitable for Gaussian emissions, and does it matter that *Pdgfrb* is missing from `.X`.

## 8 Plan for the next two weeks

- Read Bilmes (1998): Gaussian mixture example first, then the HMM case. Replace the discrete $B[j,k]$ in the toy code by 1D and then 2D Gaussians and check that the known means and variances are recovered.
- Add several random restarts to the toy Baum–Welch and keep the best log-likelihood; check parameter recovery after matching the state labels.
- Open `adata_final.h5ad`, check the normalisation, and compute the state proportions per time point.
- Write down the population-level likelihood and its E-step and M-step with the supervisor before fitting anything to the real data.
- Set up the validation agreed in the meeting: simulate cells from a fitted model and compare, gene by gene, the simulated and the real distribution of expression values.

## A Code: `hmm.py`

The toy implementation discussed in Sections 1–4: scaled forward and backward passes, the posteriors $\gamma$ and $\xi$, Baum–Welch re-estimation, and the consistency checks. The file is included verbatim from `project-code/hmm.py`.

```

import numpy as np

# Shape conventions
#   N = number of hidden states   (A is N x N)
#   M = number of symbols         (B is N x M)
#   T = sequence length           (observations has length T)
#   pi: (N,)   A: (N, N)   B: (N, M)
#   alpha, beta, gamma: (T, N)    xi: (T-1, N, N)


STATES = ["S1", "S2", "S3"]
SYMBOLS = ["A", "B", "C", "D"]

pi = np.array([0.6, 0.3, 0.1])  # Initial state probabilities

A = np.array([
    [0.7, 0.2, 0.1],  # Transition probabilities from S1
    [0.1, 0.3, 0.6],  # Transition probabilities from S2
    [0.2, 0.3, 0.5]   # Transition probabilities from S3
])

B = np.array([
    [0.7, 0.1, 0.1, 0.1],  # Emission probabilities for S1
    [0.1, 0.7, 0.1, 0.1],  # Emission probabilities for S2
    [0.1, 0.1, 0.4, 0.4]   # Emission probabilities for S3
])


def validate_hmm_parameters(pi, A, B):
    """
    Check that pi, A and B agree on the number of states and that every
    probability row sums to 1. N is taken from pi (one entry per state).
    """
    N = len(pi)
    M = len(SYMBOLS)
    assert pi.ndim == 1 and N > 0, "pi must be a non-empty 1-D array"
    assert A.shape == (N, N), "A must be N x N"
    assert B.shape == (N, M), "B must be N x M (one row per state, one column per symbol)"
    assert np.isclose(np.sum(pi), 1), "Initial state probabilities must sum to 1."
    assert np.allclose(A.sum(axis=1), 1), "Transition probabilities must sum to 1 for each state."
    assert np.allclose(B.sum(axis=1), 1), "Emission probabilities must sum to 1 for each state."


validate_hmm_parameters(pi, A, B)


def pick_from_row(probability_row, random_generator):
    """
    Return the zone dart landed in based on the given probability row and a random number generator.
    """
    cumulative_sum = np.cumsum(probability_row) # Return the array of cumulative sums of the probability row
    random_value = random_generator.random()
    return np.searchsorted(cumulative_sum, random_value)


def sample_hmm_sequence(pi, A, B, T, random_generator):
    """
    Sample a sequence of states and observations from the HMM defined by
    the initial state probabilities (pi), transition probabilities (A), and emission probabilities (B).
    """
    states = np.zeros(T, dtype=int)
    observations = np.zeros(T, dtype=int)

    # Sample the initial state
    first_state = pick_from_row(pi, random_generator)
    states[0] = first_state
    observations[0] = pick_from_row(B[first_state], random_generator)

    # Sample the rest of the states and observations
    for t in range(1, T):
        current_state = states[t - 1]
        next_state = pick_from_row(A[current_state], random_generator)
        states[t] = next_state
        observations[t] = pick_from_row(B[next_state], random_generator)

    return states, observations

def forward_algorithm(pi, A, B, observations):
    """
    Compute the forward probabilities (alpha) for a sequence of observations.
    alpha[t, i] is the probability of being in state i at time t, given the observations up to time t.

    Every row of alpha is divided by its sum so the numbers do not shrink to zero on long sequences.
    The sum that was divided out at time t is saved in scale_factors[t].
    The likelihood is the product of all scale factors, so the log-likelihood is the sum of their logs.

    Returns alpha (T x N, each row sums to 1), scale_factors (length T) and log_likelihood.
    """
    T = len(observations)
```

```
    N = len(pi)
    alpha = np.zeros((T, N))
    scale_factors = np.zeros(T)

    # Initialise: alpha_1(i) = pi_i * B[i, o_1], then divide the row by its sum
    alpha_row = pi * B[:, observations[0]]
    scale_factors[0] = alpha_row.sum()
    alpha[0] = alpha_row / scale_factors[0]

    # Recurse: alpha_t(j) = (sum_i alpha_{t-1}(i) * A[i, j]) * B[j, o_t], then divide the row by its sum
    for t in range(1, T):
        alpha_row = np.zeros(N)
        for j in range(N):
            transition_sum = alpha[t-1] @ A[:, j]  # dot product = sum_i alpha[t-1, i] * A[i, j]
            alpha_row[j] = transition_sum * B[j, observations[t]]
        scale_factors[t] = alpha_row.sum()
        alpha[t] = alpha_row / scale_factors[t]

    # Terminate: log P(O) = sum_t log c_t
    log_likelihood = np.sum(np.log(scale_factors))
    return alpha, scale_factors, log_likelihood


def backward_algorithm(pi, A, B, observations, scale_factors):
    """
    Compute the backward probabilities (beta) for a sequence of observations.
    beta[t, i] is the probability of the observations after time t, given state i at time t.

    Row t of beta is divided by scale_factors[t+1], the same number the forward pass divided out at time t+1.
    This keeps alpha and beta on the same scale, so alpha[t] * beta[t] sums to 1 at every t.

    Returns beta (T x N) and log_likelihood. The log-likelihood must match the one from the forward pass.
    """
    T = len(observations)
    N = len(pi)
    beta = np.zeros((T, N))
    beta[T-1] = 1  # Initialise: beta_T(i) = 1 for all i

    # Recurse backwards: beta_t(i) = sum_j A[i, j] * B[j, o_{t+1}] * beta_{t+1}(j), then divide by c_{t+1}
    for t in range(T-2, -1, -1):
        for i in range(N):
            beta[t, i] = np.sum(A[i, :] * B[:, observations[t+1]] * beta[t+1])
        beta[t] /= scale_factors[t+1]

    # Terminate: P(O) = sum_i pi_i * B[i, o_1] * beta_1(i). beta[0] was scaled by c_2 .. c_T, so add those logs back
    first_step = np.sum(pi * B[:, observations[0]] * beta[0])
    log_likelihood = np.log(first_step) + np.sum(np.log(scale_factors[1:]))
    return beta, log_likelihood


def posteriors(pi, A, B, observations, alpha, beta, scale_factors):
    """
    Compute the posteriors gamma and xi from alpha and beta. This is the E-step.
    gamma[t, i] is the probability of being in state i at time t, given the whole sequence.
    xi[t, i, j] is the probability of moving from state i at time t to state j at time t+1, given the whole sequence.

    alpha and beta share the same scaling, so gamma = alpha * beta already sums to 1 in every row.
    xi is divided by scale_factors[t+1] so every t-slice sums to 1.

    Returns gamma (T x N) and xi ((T-1) x N x N).
    """
    T = len(observations)
    N = len(pi)

    gamma = alpha * beta

    xi = np.zeros((T-1, N, N))
    for t in range(T-1):
        for i in range(N):
            for j in range(N):
                xi[t, i, j] = alpha[t, i] * A[i, j] * B[j, observations[t+1]] * beta[t+1, j]
        xi[t] /= scale_factors[t+1]

    return gamma, xi


def e_step(pi, A, B, observations):
    """
    E-step of Baum-Welch: forward pass, backward pass, then the posteriors gamma and xi
    under the current parameters pi, A and B.

    Returns gamma (T x N), xi ((T-1) x N x N) and the log-likelihood log P(O | lambda).
    """
    alpha, scale_factors, log_likelihood = forward_algorithm(pi, A, B, observations)
    beta, _ = backward_algorithm(pi, A, B, observations, scale_factors)
    gamma, xi = posteriors(pi, A, B, observations, alpha, beta, scale_factors)
    return gamma, xi, log_likelihood


def m_step(observations, gamma, xi, M):
    """
    Baum-Welch re-estimation. Turn the posteriors gamma and xi into new parameters.
    Every new parameter is a normalised expected count:
      pi_i    <- gamma_1(i)
      A[i, j] <- sum_{t=1}^{T-1} xi_t(i, j)  /  sum_{t=1}^{T-1} gamma_t(i)
```

```
      B[j, k] <- sum_{t : o_t = k} gamma_t(j) /  sum_t gamma_t(j)

    Returns new pi (N,), A (N x N) and B (N x M).
    """
    N = gamma.shape[1]

    # pi: posterior of the first state
    new_pi = gamma[0].copy()

    # A: expected transitions i -> j over expected visits to i (last time step excluded, no transition leaves it)
    expected_transitions = xi.sum(axis=0)              # (N, N)
    expected_visits = gamma[:-1].sum(axis=0)           # (N,)
    new_A = expected_transitions / expected_visits[:, None]

    # B: expected emissions of symbol k from state j over expected visits to j
    new_B = np.zeros((N, M))
    for k in range(M):
        new_B[:, k] = gamma[observations == k].sum(axis=0)
    new_B /= gamma.sum(axis=0)[:, None]

    return new_pi, new_A, new_B


def baum_welch(pi, A, B, observations, max_iterations=100, tolerance=1e-6):
    """
    Fit pi, A and B to the observations by repeating e_step and m_step
    until the log-likelihood stops increasing by more than tolerance.

    Returns the fitted pi, A, B and the list of log-likelihoods, one per iteration.
    """
    M = B.shape[1]
    log_likelihoods = []
    for _ in range(max_iterations):
        gamma, xi, log_likelihood = e_step(pi, A, B, observations)
        log_likelihoods.append(log_likelihood)

        pi, A, B = m_step(observations, gamma, xi, M)

        if len(log_likelihoods) > 1 and log_likelihoods[-1] - log_likelihoods[-2] < tolerance:
            break

    return pi, A, B, log_likelihoods


if __name__ == "__main__":
    states, observations = sample_hmm_sequence(pi, A, B, 5, np.random.default_rng(0))
    observations = np.array([SYMBOLS.index(s) for s in ["B", "C", "C", "A", "A"]])  # Override with a fixed observation sequence
      for testing
    print("states:", states)
    print("observations:", observations)
    alpha, scale_factors, log_likelihood_forward = forward_algorithm(pi, A, B, observations)
    beta, log_likelihood_backward = backward_algorithm(pi, A, B, observations, scale_factors)
    gamma, xi = posteriors(pi, A, B, observations, alpha, beta, scale_factors)

    print("alpha:\n", alpha)
    print("beta:\n", beta)
    print("gamma:\n", gamma)
    print("xi:\n", xi)

    print("log-likelihood (forward):", log_likelihood_forward)
    print("log-likelihood (backward):", log_likelihood_backward)

    # Forward and backward checks: two ways, same number
    assert np.isclose(log_likelihood_forward, log_likelihood_backward)
    assert np.allclose(alpha.sum(axis=1), 1)
    assert np.allclose((alpha * beta).sum(axis=1), 1)

    # Posterior checks
    assert np.allclose(gamma.sum(axis=1), 1)
    assert np.allclose(xi.sum(axis=(1, 2)), 1)
    assert np.allclose(xi.sum(axis=2), gamma[:-1])  # summing xi over j gives gamma

    decoded_states = gamma.argmax(axis=1)
    accuracy = np.mean(decoded_states == states)
    print(f"log-likelihood = {log_likelihood_forward:.3f}")
    print(f"argmax(gamma) matches the true state at {accuracy:.1%} of time steps")

    # M-step checks: one re-estimation from the true parameters gives valid distributions
    new_pi, new_A, new_B = m_step(observations, gamma, xi, len(SYMBOLS))
    validate_hmm_parameters(new_pi, new_A, new_B)

    # Baum-Welch on a long sequence: start from a random guess, the log-likelihood must never decrease
    _, long_observations = sample_hmm_sequence(pi, A, B, 2000, np.random.default_rng(0))
    rng = np.random.default_rng(1)
    pi_guess = rng.dirichlet(np.ones(len(STATES)))
    A_guess = rng.dirichlet(np.ones(len(STATES)), size=len(STATES))
    B_guess = rng.dirichlet(np.ones(len(SYMBOLS)), size=len(STATES))
    pi_fit, A_fit, B_fit, log_likelihoods = baum_welch(pi_guess, A_guess, B_guess, long_observations)
    validate_hmm_parameters(pi_fit, A_fit, B_fit)
    assert np.all(np.diff(log_likelihoods) >= -1e-8), "log-likelihood decreased during Baum-Welch"

    _, _, log_likelihood_true = forward_algorithm(pi, A, B, long_observations)
    print(f"Baum-Welch on {len(long_observations)} observations: {len(log_likelihoods)} iterations, "
          f"log-likelihood {log_likelihoods[0]:.1f} -> {log_likelihoods[-1]:.1f} "
          f"(true parameters: {log_likelihood_true:.1f})")
```

```
    # Note: the fitted states may come out in a different order than S1, S2, S3 (label switching)
    np.set_printoptions(precision=3, suppress=True)
    print("fitted A =\n", A_fit)
    print("fitted B =\n", B_fit)
```